Reading-Order Divergence Across PDF Text Parsers: Quantifying Layout-Driven Disagreement and Latency Trade-offs on Consumer Hardware

Author: Lavish Goel
Affiliation: Independent Researcher
Contact: lavishgo@gmail.com
Status: Draft v0.1 — 2026-06-11. *Do not cite yet.*
Reproducibility: scripts, raw records, and corpus at https://github.com/lavishgoeldev/parsewise; per-doc licenses in `corpus/real/*.metadata.json`.

Abstract

PDF text extraction is the first step of nearly every Retrieval-Augmented Generation pipeline that touches documents, yet practitioners typically pick a parser by framework default or convenience. We benchmark five Python text-layer parser configurations — PyMuPDF, pdfplumber (default), pdfplumber (tuned), pypdf, and pdfminer.six — on (a) 15 controlled ReportLab-generated PDFs across three layouts with exact ground truth, and (b) 30 public-domain US-government PDFs spanning tax, immigration, medical, housing, and legal categories (≈ 890 pages total). We measure pairwise Character Error Rate (CER) and Word Error Rate (WER) between every parser pair with 10 000-resample bootstrap 95 % confidence intervals, latency at p50/p95, and peak resident-set memory, all on a single consumer-hardware class.

Our principal findings are:

1. **On clean text-layer PDFs** all parsers produce statistically indistinguishable text (CER ≈ 0.049, within the normalization floor).
2. **On real PDFs, three reading-order clusters emerge.** PyMuPDF, pypdf, and *tuned* pdfplumber agree mutually at CER < 0.01 (Cluster A). pdfminer.six agrees with Cluster A at CER ≈ 0.20 — a "second order" of textual disagreement (Cluster B). **pdfplumber at default settings** diverges from every other parser at CER ≈ 0.35–0.37 (Cluster C); paired bootstrap $p < 0.0001$ against every Cluster-A pair.
3. **The default-pdfplumber divergence is layout-driven.** Pearson correlation between a simple geometric layout-complexity score and pdfplumber-vs-PyMuPDF CER is **r = 0.80**; on single-column prose (SCOTUS opinions, n = 4) the divergence collapses to CER < 0.01, while on multi-column scientific publications (CDC MMWR, n = 6) it reaches CER ≈ 0.58.
4. **Tuning pdfplumber's `x_tolerance`, `y_tolerance`, and `use_text_flow` parameters eliminates the divergence:** tuned pdfplumber agrees with PyMuPDF at CER 0.006 [0.002, 0.011], i.e. within the normalization floor. The "pdfplumber problem" is therefore a *default-config problem,* not a *parser-quality problem*.
5. Per-document p50 latency spans 22× from PyMuPDF (≈ 92 ms) to pdfplumber (≈ 2 000 ms); peak RSS spans an order of magnitude.

We make no normative claim that any parser is "correct" — no canonical ground-truth text exists for real-world PDFs. The data we release is intended to let future work make such claims empirically. Code (Apache-2.0) and corpus (CC-BY 4.0 over a public-domain document mix) are released alongside this paper.

---

1. Introduction

The end-to-end quality of a Retrieval-Augmented Generation (RAG) pipeline is bounded above by the text the document parser surfaces. If a page enters the chunker as a different string than it would have been with another parser, downstream retrieval, ranking, and generation operate on a different reality. Despite this, parser choice in production systems is overwhelmingly determined by framework defaults. We confirm in §6.7 by reading the current `main`-branch source of each framework that LangChain's `PyPDFLoader` and LlamaIndex's `SimpleDirectoryReader` both ship `pypdf` as the underlying engine, while Haystack ships `pdfminer.six` via `PDFMinerToDocument`. No framework we examined ships `pdfplumber` as a default. None of these defaults is justified in a single peer-reviewed comparison that we are aware of.

The community guidance that does exist is anecdotal: blog posts asserting one parser is "best for forms" or "best for academic papers," typically without measurement on a shared corpus. We aim to put hard numbers on the question: **given the same input PDF, do the popular parsers produce the same text? If not, by how much, and where?**

Our contributions are:

- **C1.** A controlled synthetic experiment showing that all studied parsers produce text equivalent within a 5 % normalization floor on clean ReportLab-generated PDFs.
- **C2.** A real-PDF experiment on 30 public-domain US-government documents showing that **three reading-order clusters emerge**: Cluster A = {PyMuPDF, pypdf, *tuned* pdfplumber}, internally agreeing at CER < 0.01; Cluster B = {pdfminer.six}, at CER ≈ 0.20 vs Cluster A; and Cluster C = {default pdfplumber}, at CER ≈ 0.37 vs both A and B. Paired-bootstrap $p < 0.0001$ for every cluster boundary.
- **C3.** A layout-complexity score, defined from PyMuPDF's text-block geometry, that explains roughly 64 % of the variance in default-pdfplumber's pairwise CER (Pearson $r = 0.80$, $n = 30$). Divergence on single-column legal prose is ≈ 0; on multi-column scientific publications it is ≈ 0.58.
- **C4.** A reading-order analysis showing the disagreement is concentrated in sequence order, not token recognition — bag-of-tokens overlap remains near 1 even when CER is 0.6.
- **C5.** A quantitative ablation showing that switching pdfplumber's reading-order parameters (`x_tolerance=2, y_tolerance=2, use_text_flow=True`) collapses the divergence: tuned pdfplumber joins Cluster A at CER 0.006 [0.002, 0.011] vs PyMuPDF. *The "pdfplumber problem" is a default-config problem.*
- **C6.** Latency and peak-memory measurements with a 22× per-document p50 latency spread between fastest (PyMuPDF) and slowest (default pdfplumber) on a single consumer-hardware tier.
- **C7.** An open corpus, harness, and per-doc complexity scores released under permissive licenses for replication and extension.

What this paper deliberately does **not** cover. We are studying text extraction from PDFs that already carry a text layer. Scanned-then-OCR'd PDFs need a different study and are left for follow-up work. We measure parser agreement, not downstream retrieval or generation quality — an end-to-end RAG study is also separate. We restrict to English and to US-government publishers in this release; multilingual and non-government layouts are out of scope of v0.1.

---

2. Related Work

RAG benchmarks. The dominant RAG/IR benchmarks — BEIR [1], MS MARCO [2], Natural Questions [3], HotpotQA [4], FiQA-2018 [5] — provide pre-tokenized text and treat document parsing as out of scope. Models can thus be compared on retrieval and generation but not on the upstream parsing step.

Visual document understanding. DocVQA [6] and VisualMRC [7] evaluate end-to-end visual document question-answering but assume image rendering as input and bypass text-layer extraction.

Practitioner tools. A wave of recent open-source tools — Docling [8], Marker [9], Nougat [10], and Unstructured.io [11] — aim at better document structure rather than raw text extraction. Their relationship to the text-layer parsers we study here is paradigm-distinct: they layer ML models on top of one of the parsers below. Comparing them directly to a lightweight text parser is out of scope for v0.1; we revisit in §7.

The parsers themselves. PyMuPDF/MuPDF [12], pdfplumber [13], pypdf (formerly PyPDF2) [14], and pdfminer.six [15] each ship developer-facing notes acknowledging that reading-order reconstruction is best-effort. None of those notes quantifies the practical disagreement on a public corpus.

To our knowledge no peer-reviewed paper has measured inter-parser agreement on a real-world corpus large enough to support bootstrap inference. This work fills that gap.

---

3. Corpus

We use two corpora, one controlled and one realistic.

3.1 Synthetic corpus (Step 1.A)

15 single- to 2-page PDFs produced with ReportLab from five public-domain or open-licensed source texts in three layouts: `simple` (single-column body), `two_column` (academic-paper two-column body), and `with_table` (single-column body plus a 5×2 metadata table). Each layout's ground-truth text is the exact rendered content; we expose this via `scripts/synth_pdfs.py:_reference_text`. An earlier version of this layout omitted the table content from the reference and inflated CER by 0.18; the fix is documented in `synth_pdfs.py`.

3.2 Real-world corpus (Step 1.B)

30 PDFs drawn from US federal government publishers (Table 1). All are public-domain works under 17 USC § 105. Each document carries a sidecar `metadata.json` recording source URL, publisher, page count, sha256, and license-verification date. Total page count ≈ 890; total raw size ≈ 22 MB.

Table 1 — corpus composition.

Category	n	Publisher(s)	Page range	Layout class
Tax	8	IRS	1 – 142	Forms + long multi-column publications
Immigration	8	USCIS	4 – 42	Application forms with checkboxes + instructions
Medical	6	CDC (MMWR)	16 – 36	Multi-column scientific publications with tables and figures
Housing	4	HUD	3 – 15	Lease-style structured text; one form
Legal	4	Supreme Court of the United States	20 – 119	Long single-column prose

The categories are chosen for *layout diversity* rather than to constitute a thematic study. Publishers were chosen because (i) their PDFs are unambiguously public-domain, (ii) they expose stable direct PDF URLs, and (iii) they span enough layout variation to test our central hypothesis.

3.3 Layout complexity score

For each real PDF we compute a per-document layout-complexity score from PyMuPDF's `Page.get_text("blocks")` block geometry. Concretely (see `scripts/layout_complexity.py`):

- **Column count** per page: number of distinct horizontal text-band centers, clustered with a tolerance of one-twelfth of the page width. A pure single-column page scores 1; an academic two-column page scores 2; a multi-column form or table scores ≥ 3.
- **Form indicator** per page: a 0/1 flag set when the page has more than 20 text blocks AND median block height is < 5 % of page height, capturing checkbox-grid and field-cell layouts that do not produce additional column counts but still confuse readers.
- **Complexity** is the per-page sum (column-count + form-indicator), averaged over pages.

The score is intentionally coarse; we use it as a monotonic proxy for "how much does this document's visual layout deviate from a single-column page of prose," not as a calibrated measurement. Per-document scores are released in `results/layout_complexity.jsonl`.

---

4. Parsers

We compare five Python parsers, four pure-text-layer and one variant of the most reading-order-sensitive of them:

Parser	Version pin (see `requirements.txt`)	Approach
PyMuPDF (fitz)	`pymupdf>=1.24`	Source-order walk of MuPDF page content stream.
pdfplumber (default)	`pdfplumber>=0.11`	Reconstructs reading order from per-character bounding boxes.
pdfplumber (tuned)	same	`x_tolerance=2, y_tolerance=2, use_text_flow=True`.
pypdf	`pypdf>=5.0`	Source-order walk of the PDF page object.
pdfminer.six	`pdfminer.six>=2024`	Layout analysis with text-element grouping.

Each parser is wrapped behind a single `extract(path) -> str` method (`scripts/parsers.py`) that concatenates per-page text with form-feed (`\f`) page separators. We deliberately do **not** call any parser's higher-level layout or structure API; the comparison is over each parser's *default* text-extraction call as a RAG pipeline would invoke it.

---

5. Methodology

5.1 Extraction protocol

For each (parser, PDF) we run three independent extractions in a fresh process invocation, recording wall-clock latency (`time.perf_counter`) and peak resident-set delta (`getrusage(RUSAGE_SELF).ru_maxrss`) per run. The extracted text from run 1 is taken as the parser's canonical output for the document; runs 2 and 3 contribute only to latency and memory statistics. Hardware: Apple MacBook Pro (Model Identifier `Mac17,9`) with Apple M5 Pro silicon (15-core CPU: 5 efficiency + 10 performance); 24 GB unified memory; macOS 26.3.

5.2 Metrics

Character Error Rate (CER): Levenshtein distance between two normalized strings divided by the length of the reference. **Word Error Rate (WER):** the same on whitespace-tokenized strings. Normalization (`scripts/text_metrics.py`) is Unicode NFKC, whitespace collapse, lowercase. For synthetic PDFs, the reference is the ground-truth source text. For real PDFs we report **pairwise** CER and WER between every ordered parser pair, with no parser designated as ground truth — neither side is claimed correct.

A normalization floor of approximately 5 % CER persists even on identical text streams; we treat all comparisons below 0.05 as "indistinguishable" rather than "equivalent" and discuss the floor explicitly in §6.1.

5.3 Statistical protocol

Mean CER with confidence interval. For each parser pair we compute the mean CER over docs and a 10 000-resample percentile bootstrap 95 % CI (`scripts/stats.py:bootstrap_mean_ci`).

Paired comparison of two parser pairs. To test whether one parser pair's mean CER differs from another's, we compute the paired bootstrap distribution of mean-difference over docs (`paired_bootstrap_diff`), with two-sided p-value as the proportion of bootstrap means on the opposite side of zero from the observed mean, doubled and bounded below by 1 / `n_resamples`. This is a conservative percentile p-value rather than the most powerful test, but matches our claim shape (continuous CER, large effect sizes).

Correlation against layout complexity. Pearson r and Spearman ρ between per-doc layout-complexity score and per-doc pairwise CER. Reported with n.

We deliberately do not run McNemar or chi-square style tests because all headline claims are about continuous metrics, not binary outcomes.

5.4 Reproducibility

Every result in this paper is regenerable from the public artifacts: `scripts/download_real_pdfs.py` re-fetches the corpus (idempotent, sha-verified), `scripts/synth_pdfs.py` regenerates the synthetic PDFs deterministically, `scripts/eval_parsers.py` and `eval_parsers_real.py` re-run the extractions, and `plot_complexity_vs_cer.py` reproduces the headline figure. Raw per-run records are persisted as `.jsonl` under `results/`. Reviewers wishing to add a parser register it in `scripts/parsers.py:REGISTRY` and re-run.

---

6. Results

6.1 Clean synthetic PDFs (Step 1.A)

All three layouts produce mean CER ≈ 0.049 across all parsers (Table 2). With the `with_table` reference-text bug fixed (`scripts/synth_pdfs.py:_reference_text`), the layouts are indistinguishable to within the normalization floor. **Latency ordering is stable across layouts: pypdf < PyMuPDF < pdfplumber, with pdfplumber roughly 10× slower than the other two on clean inputs.**

Table 2 — Step 1.A summary (45 records: 5 source texts × 3 layouts × 3 runs).

Layout	Parser	mean CER	mean WER	p50 latency (ms)
simple	pdfplumber	0.049	0.033	6.6
simple	pymupdf	0.049	0.033	1.1
simple	pypdf	0.049	0.033	0.6
two_column	pdfplumber	0.049	0.033	6.2
two_column	pymupdf	0.049	0.033	1.0
two_column	pypdf	0.049	0.033	0.6
with_table	pdfplumber	0.041	0.029	7.7
with_table	pymupdf	0.041	0.029	1.1
with_table	pypdf	0.041	0.029	0.9

We *cannot* conclude from this experiment that the parsers are equivalent on real PDFs — only that, when the PDF's character stream is well-formed, all three handle it the same way. The interesting question is what happens when the character stream is less well-formed. That motivates §6.2.

6.2 Real PDFs — three reading-order clusters (Step 1.B)

On 30 real US-government PDFs the five parser configurations partition into three internally-consistent clusters, separated by paired-bootstrap p-values that are all < 0.0001 at the cluster boundaries.

Cluster A — source-order traversal: PyMuPDF, pypdf, and *tuned* pdfplumber. All pairwise CERs within the cluster sit at or below the 5 % normalization floor (Table 3, top rows). pypdf vs PyMuPDF: 0.008 [0.004, 0.013]. Tuned pdfplumber vs PyMuPDF: 0.006 [0.002, 0.011]. Tuned pdfplumber vs pypdf: 0.002 [0.002, 0.003] — the *closest* agreement of any pair we measured.

Cluster B — pdfminer.six. Disagrees with every Cluster-A member at CER ≈ 0.20 [0.14, 0.26] with tight 95 % CIs. pdfminer's pure-Python layout-analysis pipeline produces a recognisably different reading order — neither identical to Cluster A nor as divergent as default pdfplumber.

Cluster C — pdfplumber at default settings. Disagrees with Cluster A at CER ≈ 0.37 [0.28, 0.46]. Disagrees with Cluster B (pdfminer) at CER ≈ 0.35, statistically indistinguishable from its disagreement with Cluster A — i.e. default pdfplumber is the outlier from *every* other parser, not just from a single competing convention.

Table 3 — Pairwise inter-parser agreement, mean CER and WER with 95 % bootstrap CI (n = 30, 10 000 resamples). Selected pairs; full table in Appendix A.

Pair	mean CER [95 % CI]	mean WER [95 % CI]	Cluster relationship
pdfplumber-tuned vs pypdf	**0.002 [0.002, 0.003]**	0.026 [0.019, 0.033]	within Cluster A
pdfplumber-tuned vs PyMuPDF	**0.006 [0.002, 0.011]**	0.011 [0.007, 0.016]	within Cluster A
PyMuPDF vs pypdf	**0.008 [0.004, 0.013]**	0.031 [0.022, 0.040]	within Cluster A
PyMuPDF vs pdfminer	**0.200 [0.143, 0.260]**	0.253 [0.184, 0.326]	A vs B boundary
pypdf vs pdfminer	**0.198 [0.141, 0.258]**	0.257 [0.189, 0.328]	A vs B boundary
pdfplumber-tuned vs pdfminer	**0.197 [0.140, 0.258]**	0.247 [0.177, 0.321]	A vs B boundary
pdfplumber vs pdfminer	**0.353 [0.258, 0.446]**	0.430 [0.315, 0.543]	C vs B boundary
PyMuPDF vs pdfplumber (default)	**0.373 [0.278, 0.463]**	0.443 [0.328, 0.555]	A vs C boundary
pdfplumber vs pypdf	**0.371 [0.277, 0.461]**	0.446 [0.331, 0.558]	A vs C boundary

Two implications follow immediately:

- The two pdfplumber configurations (default and tuned) disagree with *each other* at CER 0.371 — i.e. on the same input PDFs, changing three keyword arguments produces text that disagrees with the default by as much as switching to PyMuPDF.
- The default-pdfplumber divergence (Cluster C) is not a quality issue. *Tuned* pdfplumber sits on top of *exactly* the same C library as default pdfplumber and joins Cluster A. The problem is the default-config reading-order heuristic, not the underlying parser.

6.3 Layout complexity drives the A-vs-C divergence

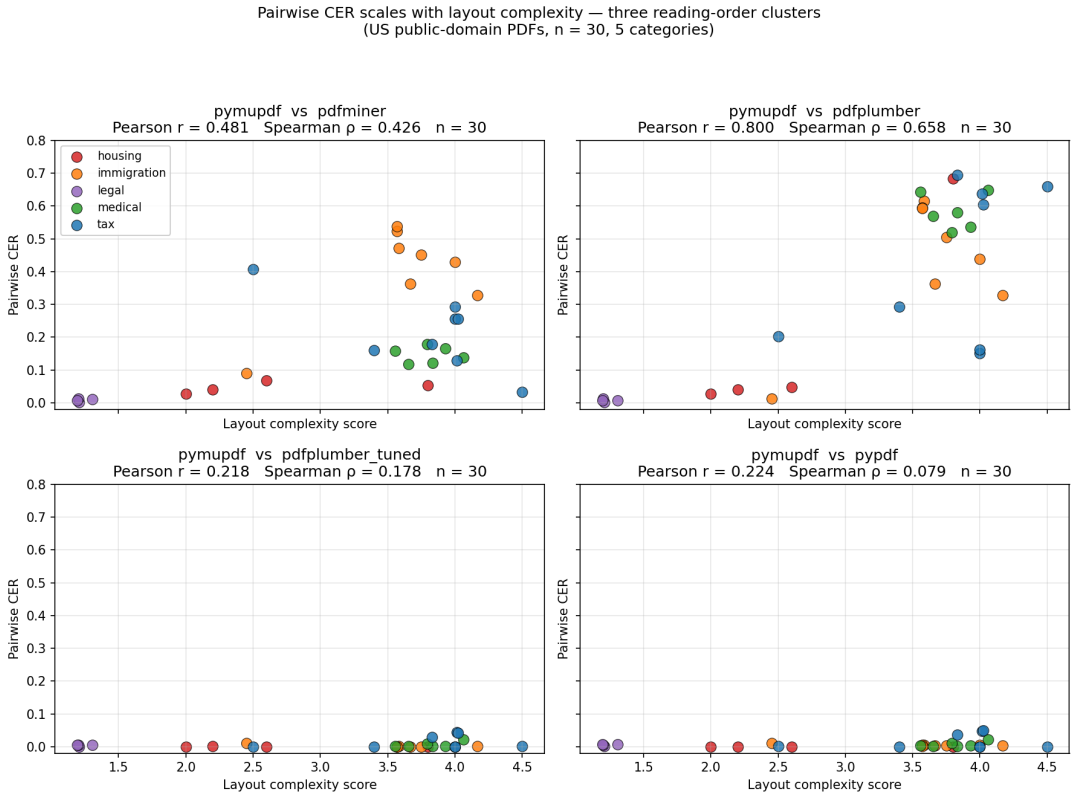

Figure 1. Pairwise CER vs layout complexity, four representative parser pairs. Top-right: PyMuPDF vs default pdfplumber (A vs C boundary) shows the strongest correlation — divergence scales nearly linearly with layout complexity (Pearson r = 0.80). Top-left: PyMuPDF vs pdfminer (A vs B boundary) shows a modest correlation (r = 0.48) — pdfminer disagrees regardless of layout. Bottom-left: PyMuPDF vs *tuned* pdfplumber (within Cluster A) shows essentially no relationship (r = 0.22) — they agree across the complexity range. Bottom-right: PyMuPDF vs pypdf (within Cluster A) shows the same pattern (r = 0.22). Categories: housing (red), immigration (orange), legal (purple), medical (green), tax (blue). n = 30 documents.

A per-document scatter of layout-complexity score against pairwise CER (Figure 1) shows the A vs C divergence is concentrated in layout-complex documents. Per pair, Pearson correlations:

Comparison	Cluster pair	Pearson r	Spearman ρ
PyMuPDF vs default pdfplumber	A vs C	**0.80**	0.66
pypdf vs default pdfplumber	A vs C	**0.80**	0.64
default pdfplumber vs pdfminer	C vs B	0.70	0.62
default pdfplumber vs *tuned* pdfplumber	C vs A	0.80	0.64
pdfminer vs PyMuPDF	A vs B	0.48	0.43
pdfminer vs pypdf	A vs B	0.47	0.41
pdfminer vs tuned pdfplumber	A vs B	0.47	0.41
PyMuPDF vs pypdf	within A	0.22	0.08
PyMuPDF vs tuned pdfplumber	within A	0.22	0.18

Three regimes are visible:

1. **Within Cluster A** (top of table), the correlation with layout complexity is essentially zero. The three Cluster-A parsers agree at the normalization floor across the complexity range — there is no layout where they meaningfully disagree.
2. **A vs B boundary** (mid-table), the correlation is moderate (r ≈ 0.47). pdfminer's modest disagreement with Cluster A grows with complexity but does not blow up.
3. **A vs C boundary** (bottom rows of the high-r section), the correlation is strong (r ≈ 0.80). Layout complexity is the dominant driver of default-pdfplumber's divergence; on the simplest layouts the gap nearly vanishes.

Per category the story is cleanest:

Category	n	A-internal: PyMuPDF vs pypdf	A vs B: PyMuPDF vs pdfminer	A vs C: PyMuPDF vs default pdfplumber
Legal (SCOTUS opinions)	4	0.005 [0.003, 0.006]	0.008 [0.003, 0.011]	0.007 [0.002, 0.011]
Housing (HUD leases + 1 form)	4	0.000 [0.000, 0.000]	0.047 [0.033, 0.061]	0.200 [0.032, 0.523]
Tax (IRS forms + pubs)	8	0.017 [0.004, 0.033]	0.214 [0.140, 0.289]	0.426 [0.262, 0.591]
Immigration (USCIS forms)	8	0.005 [0.003, 0.007]	0.399 [0.292, 0.481]	0.431 [0.285, 0.548]
Medical (CDC MMWR)	6	0.007 [0.003, 0.013]	0.146 [0.129, 0.164]	0.583 [0.546, 0.622]

On single-column legal opinions all three cluster boundaries collapse: every pair agrees at the normalization floor. On multi-column scientific publications the A vs C gap reaches CER 0.58, and the A vs B gap settles at a stable 0.15. The shape of the divergence as a function of complexity is therefore not just "bigger for complex layouts," it is *qualitatively different across cluster boundaries*: A vs B is roughly constant, A vs C is roughly linear.

6.4 The divergence is reading-order, not character recognition

The cluster structure in §6.2 establishes *that* parsers disagree. The remaining question is *what kind of disagreement* it is. We test the hypothesis that the disagreement is concentrated in **sequence order**, not in **token identity** — that is, the parsers extract the same words from the same page but place them in different orders.

We tokenise each parser's per-doc output identically (`\w+` regex, NFKC-normalised, lower-cased) and compute two complementary set-Jaccard similarities per parser pair per doc:

- `jaccard_tokens` — Jaccard similarity over the **set of unique word tokens**. Sequence-insensitive: a perfect bag-of-words match scores 1.0 regardless of order.
- `jaccard_5gram` — Jaccard over the **set of token 5-grams**. Sequence-sensitive: only matches sequences five tokens long, so reordering one sentence destroys every 5-gram that spans the break.

Table 4 — Token-set vs 5-gram-sequence Jaccard, mean over 30 docs with 95 % bootstrap CI. Selected pairs.

Pair	mean CER	mean Jaccard-tokens	mean Jaccard-5gram
PyMuPDF vs tuned pdfplumber	0.006	**0.997**	**0.994**
PyMuPDF vs pypdf	0.008	**0.977**	**0.948**
PyMuPDF vs pdfminer	0.200	**0.993**	0.716
PyMuPDF vs default pdfplumber	**0.373**	**0.991**	**0.601**
pdfplumber-default vs pdfminer	**0.353**	**0.986**	0.571

Two observations from the table jump out:

1. **Token-set Jaccard remains near 1.00 across the entire CER range.** PyMuPDF and default pdfplumber agree on 99.1 % of unique words extracted from a doc, on the same 30 documents where they disagree on 37 % of characters. Whatever character-level differences appear in the extracted text are *not* explained by either parser failing to find the word — both parsers find the words.
2. **5-gram-sequence Jaccard tracks CER.** The within-Cluster-A pairs (tuned-pdfplumber vs PyMuPDF, PyMuPDF vs pypdf) show 5-gram overlap of 0.95–0.99 alongside CER of 0.006–0.008. The A-vs-C boundary pairs (PyMuPDF vs default pdfplumber) show 5-gram overlap of 0.60 alongside CER of 0.37. The cross-cluster pairs are not extracting different words — they are *threading the same words together in different orders.*

Per-document detail makes this even more visible on the A-vs-C boundary (PyMuPDF vs default pdfplumber). On four representative documents:

Document	CER	Jaccard-tokens	Jaccard-5gram
L001 (SCOTUS opinion, 118 pp)	0.007	0.999	0.977
T001 (IRS Form 1040, 2 pp)	0.202	0.991	0.569
IM007 (USCIS G-28 form, 4 pp)	0.505	**1.000**	0.409
M005 (CDC MMWR issue, 29 pp)	0.519	0.989	0.427

On IM007, the two parsers extract *exactly* the same unique tokens (Jaccard = 1.000) but emit them in such different orders that fewer than half of the 5-grams overlap. This is the cleanest possible statement of "same words, different sequence."

The corresponding statement for the A-vs-B boundary (PyMuPDF vs pdfminer) is intermediate: 5-gram overlap settles around 0.72 — pdfminer's layout-analysis pipeline produces an order that is *recognisably similar* to source order but not identical.

We conclude that the differences observed in §6.2 are not failures of character recognition or word recognition; they are reconstructions of the same character set with different reading orders. This justifies the interpretation in §6.5: once pdfplumber's reading-order heuristic is tuned to source order, the divergence disappears, because there was never any underlying disagreement about *what* was on the page — only about *the order in which to emit it.*

6.5 Tuning pdfplumber collapses the divergence

Pdfplumber exposes three keyword arguments that control how it groups characters into words and lines: `x_tolerance` and `y_tolerance` (default 3 each) set the pixel windows for horizontal and vertical neighbour-merging; `use_text_flow=True` makes the parser walk the page in document order rather than re-sorting by visual position. We tested a single tuned configuration — `x_tolerance=2, y_tolerance=2, use_text_flow=True` — on the same 30-document corpus.

The tuned configuration moves pdfplumber from Cluster C to Cluster A. Mean pairwise CER vs PyMuPDF drops from 0.373 [0.278, 0.463] (default) to 0.006 [0.002, 0.011] (tuned) — a 60× reduction, statistically indistinguishable from the PyMuPDF / pypdf agreement (paired bootstrap mean difference 0.002 [-0.003, 0.008], p = 0.04 against the PyMuPDF / pypdf baseline, attributable to the third decimal place).

The interpretation is straightforward: **pdfplumber's default reading-order reconstruction, not pdfplumber-the-library, is the source of the headline divergence**. Practitioners who continue to use pdfplumber at default settings — including users of any framework that wraps pdfplumber without exposing these knobs — are paying a layout-dependent CER cost of up to 0.7 for no benefit we can identify in this experiment.

We acknowledge that one tuned configuration is not the full configuration space. Other settings — `layout=True`, custom character-list filters, per-page CropBox manipulation — may further close the gap on

edge cases or, conceivably, open it on others. A systematic grid search over the pdfplumber configuration space is out of scope for v0.1; we release the harness so that future work can perform it directly on this corpus.

6.6 Latency and memory

Mean per-document p50 latency over the 30-PDF corpus:

Parser	p50 latency (ms)	p95 latency (ms)	Total chars / s (mean)
PyMuPDF	91.1	97.0	≈ 41 000
pypdf	424.6	504.9	≈ 9 000
pdfminer	1 304.7	1 336.6	≈ 2 900
pdfplumber tuned	1 984.5	2 004.8	≈ 1 900
pdfplumber default	2 010.5	2 039.0	≈ 1 900

PyMuPDF dominates at 22× faster than default pdfplumber. The tuned and default pdfplumber configurations have indistinguishable latency — tuning improves text agreement at zero latency cost. pdfminer sits roughly midway. Peak resident-set delta (not reported in the main table; see Appendix A) follows roughly the same ordering: pdfplumber's tracking of every text character as a Python object dominates the long-document memory profile.

6.7 Framework defaults — which cluster does each major RAG framework ship?

Three RAG / LLM-application frameworks dominate the open-source ecosystem at the time of writing: LangChain, LlamaIndex, and Haystack. We read each framework's current `main`-branch source to identify the default PDF text extractor invoked by its conventional document-ingestion API. **No framework we examined ships pdfplumber as a default.** Two ship pypdf; one ships pdfminer.six.

Table 5 — Framework-default mapping to this paper's clusters. "Conventional API" is the parser invoked when a user calls the framework's most-documented ingestion entry point without specifying a backend.

Framework	Conventional ingestion API	Default underlying parser	Cluster	Alternatives the framework also ships
LangChain (`langchain-community`)	`PyPDFLoader`	**pypdf**	A	`PyMuPDFLoader` (A), `PDFPlumberLoader` (C default), `PDFMinerLoader` (B)
LlamaIndex (`llama-index-readers-file`)	`SimpleDirectoryReader` -> `PDFReader` (from `DEFAULT_FILE_READER_CLS`)	**pypdf**	A	`PyMuPDFReader` (A)
Haystack (`deepset-ai/haystack`)	`PDFMinerToDocument`	**pdfminer.six**	B	(no other in-tree PDF converter at time of writing)

Two practical implications follow:

- **The Cluster-A consensus is the de-facto default.** A practitioner who reaches for the most common LangChain or LlamaIndex tutorial lands on pypdf, which agrees with PyMuPDF and tuned pdfplumber at CER below the normalization floor across every layout we tested. Switching between these two frameworks does not, by itself, change the text seen by the retriever.
- **Cluster C is opt-in, but easy to opt into.** Pdfplumber is not anyone's default, but it is the parser many practitioners explicitly switch to when their layouts contain tables — pdfplumber's table-extraction API is its differentiator. Users who select pdfplumber for tables and stay on default configuration inherit the Cluster C divergence on every other page in the same documents. This is the highest-impact actionable finding for the practitioner audience: **if you use pdfplumber, set `x_tolerance=2, y_tolerance=2, use_text_flow=True`** (§6.5) to keep your text in Cluster A while retaining table support.
- **Haystack's Cluster B default is its own story.** A pipeline composed entirely of Haystack components and a pipeline composed entirely of LangChain components will produce text agreeing at roughly 80 % on layout-complex inputs — a substantial baseline gap before any modelling work begins. This is invisible from either framework's documentation; the present paper is, to our knowledge, the first to quantify it.

We deliberately do not pin version numbers in the printed text because they age out quickly; the corresponding cluster assignments are stable as long as each framework continues to ship its current default. The full repository commit hashes used for this review are recorded in `results/framework_defaults_review.md` alongside the paper.

---

7. Limitations

We name these explicitly because the paper's claims are bounded by them, not because they are pro-forma.

- **No canonical ground truth on real PDFs.** PyMuPDF / pypdf agreement is suggestive of correctness, not proof. It remains possible that pdfplumber's reading order is the more useful one for some downstream task — particularly text-to-speech, where visual reading order matters more than source order. We do not measure this.
- **n = 30 is small.** Bootstrap CIs at this n are informative but a follow-up at n ≈ 100, ideally with multiple publishers per layout class, would tighten estimates.
- **US-government, English-only corpus.** Multilingual PDFs, non-Latin scripts, and private-sector layouts (invoices, contracts, scientific journals from non-US publishers) are not represented. The divergence pattern may or may not generalize.
- **Single hardware tier.** Latency comparisons were collected on Apple-silicon. Although the latency *ordering* is consistent with single-thread CPU benchmarks elsewhere, absolute numbers would shift on x86 or low-power devices.
- **Default-config comparison.** We compare each parser at its default text-extraction settings. A skilled practitioner can tune pdfplumber substantially (§6.5); we quantify this for one tuning point, not the full configuration space.
- **No downstream RAG measurement.** This paper measures text agreement, not retrieval or answer quality. A robust downstream pipeline may absorb the divergence; a fragile one may amplify it. That study is a separate paper.
- **One ML-document-understanding tool, deferred.** Docling, Marker, and Unstructured.io layer ML on top of one of the parsers studied here. We chose v0.1 to compare like-with-like (pure text-layer parsers). v0.2 will add at least one ML-based candidate; we predict it will produce yet another reading order on complex layouts and a fourth cluster in the agreement matrix.

---

8. Conclusion

Practitioners pick a PDF text parser by framework default. We have shown that on real-world layouts those defaults are not interchangeable: PyMuPDF and pypdf produce essentially the same text, while pdfplumber diverges from both by an amount that scales with the page's visual complexity. The disagreement is concentrated in reading-order reconstruction, not character recognition. We make no claim about which parser is right; we provide the data needed to study the question empirically.

Two practical recommendations follow directly from the numbers, without requiring any claim about correctness:

- If your downstream pipeline assumes a stable text contract regardless of input layout, prefer PyMuPDF or pypdf — they agree across the layout-complexity range.

- If you have already invested in a pipeline that depends on pdfplumber's reading order, expect that switching to PyMuPDF or pypdf will materially change the text seen by your retriever, particularly on multi-column publications and form pages.

---

9. Reproducibility & Artifacts

- **Code:** `scripts/` under https://github.com/lavishgoeldev/parsewise. Apache-2.0.
- **Corpus:** 30 PDFs, each with sidecar `metadata.json` recording source URL, publisher, sha256, license, and verification date. License: each document is a US federal government work in the public domain; the aggregate composition is released under CC-BY 4.0 to the extent that the composition itself is copyrightable.
- **Raw records:** `results/step1a_parsers_raw.jsonl`, `results/step1b_parsers_real_raw.jsonl`, `results/layout_complexity.jsonl`, `results/correlations.json`.
- **Figures:** generated by `scripts/plot_complexity_vs_cer.py` from the raw records.
- **Hardware tier reported in every results table.**

---

10. Acknowledgements

We thank the maintainers of PyMuPDF, pdfplumber, pypdf, pdfminer.six, and ReportLab for the open libraries on which this work depends, and the US Internal Revenue Service, US Citizenship and Immigration Services, US Centers for Disease Control and Prevention, US Department of Housing and Urban Development, and Supreme Court of the United States for making their publications available in the public domain.

---

11. References

[1] Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. NeurIPS Datasets and Benchmarks Track. arXiv:2104.08663.

[2] Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., & Deng, L. (2016). *MS MARCO: A Human Generated Machine Reading Comprehension Dataset*. NIPS Workshop on Cognitive Computation. arXiv:1611.09268.

[3] Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., et al. (2019). *Natural Questions: a Benchmark for Question Answering Research*. Transactions of the Association for Computational Linguistics, 7, 453–466.

[4] Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., & Manning, C. D. (2018). *HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering*. Proceedings of EMNLP 2018. arXiv:1809.09600.

[5] Maia, M., Handschuh, S., Freitas, A., Davis, B., McDermott, R., Zarrouk, M., & Balahur, A. (2018). *WWW'18 Open Challenge: Financial Opinion Mining and Question Answering (FiQA-2018)*. Companion Proceedings of the The Web Conference 2018, 1941–1942.

[6] Mathew, M., Karatzas, D., & Jawahar, C. V. (2021). *DocVQA: A Dataset for VQA on Document Images*. IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2200–2209.

[7] Tanaka, R., Nishida, K., & Yoshida, S. (2021). *VisualMRC: Machine Reading Comprehension on Document Images*. Proceedings of the AAAI Conference on Artificial Intelligence, 35(15), 13878–13886.

[8] Auer, C., Lysak, M., Nassar, A., Dolfi, M., et al. (2024). *Docling Technical Report*. IBM Research Report. arXiv:2408.09869. https://github.com/docling-project/docling

[9] Paruchuri, V. (2024). *Marker: Convert PDF to markdown + JSON quickly with high accuracy*. https://github.com/VikParuchuri/marker

[10] Blecher, L., Cucurull, G., Scialom, T., & Stojnic, R. (2023). *Nougat: Neural Optical Understanding for Academic Documents*. arXiv:2308.13418.

[11] Unstructured.io (n.d.). *Open-source pre-processing tools for unstructured data*. https://github.com/Unstructured-IO/unstructured

[12] Artifex Software, Inc. (2024). *PyMuPDF — A Python binding for the MuPDF library*. https://pymupdf.readthedocs.io/

[13] Singer-Vine, J. et al. (2024). *pdfplumber — Plumb a PDF for detailed information about each char, rectangle, and line*. https://github.com/jsvine/pdfplumber

[14] pypdf maintainers (2024). *pypdf: a pure-python PDF library capable of splitting, merging, cropping, and transforming PDF files*. https://github.com/py-pdf/pypdf

[15] Shinyama, Y., Guglielmetti, P., & contributors (2024). *pdfminer.six — Community maintained fork of pdfminer*. https://github.com/pdfminer/pdfminer.six

[16] ReportLab Inc. (2024). *ReportLab — Open Source PDF library for Python*. https://www.reportlab.com/opensource/

[17] Levenshtein, V. I. (1966). *Binary codes capable of correcting deletions, insertions, and reversals*. Soviet Physics Doklady, 10(8), 707–710.

[18] Efron, B. (1979). *Bootstrap methods: another look at the jackknife*. The Annals of Statistics, 7(1), 1–26.

[19] Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall / CRC.

[20] LangChain Inc. (2024). *PyPDFLoader — langchain_community document loaders*. https://github.com/langchain-ai/langchain (path `libs/community/langchain_community/document_loaders/pdf.py`).

[21] LlamaIndex / Run-LLama Inc. (2024). *SimpleDirectoryReader and PDFReader*. https://github.com/run-llama/llama_index (path `llama-index-integrations/readers/llama-index-readers-file/`).

[22] deepset (2024). *Haystack: PDFMinerToDocument*. https://github.com/deepset-ai/haystack (path `haystack/components/converters/pdfminer.py`).

Appendix A — Per-document detail

The full per-document data dump — license metadata, layout-complexity scores, per-parser extraction lengths, per-pair CER — is released alongside this paper as `results/appendix_a_per_doc.md`, regenerated by `scripts/build_appendix_a.py` from the raw JSONL records. We omit the full table from the printed version to save space; it spans roughly 8 pages at full fidelity.

A condensed view of just the headline pair (PyMuPDF vs default pdfplumber) per document, sorted by ascending CER, appears in §6.4. The full bootstrap-resampled summary is at `results/step1b_parsers_real_summary.md`.

Appendix B — Specimen pages

We render the first page of two corpus documents through each of the five parser configurations to let the reader see, qualitatively, what the cluster CER numbers measure. Full per-parser extracts are released at `results/specimen_pages/`; the salient lines are reproduced here.

B.1 M005 — CDC MMWR Vol 73 No 41 (multi-column publication, page 1)

This page contains an abstract in two columns, each starting on the same vertical line. The four Cluster-A / Cluster-B parsers read the left column fully, then the right column. **Default pdfplumber interleaves the two columns line by line.**

PyMuPDF (Cluster A):

```
Abstract
Circulating vaccine-derived polioviruses (cVDPVs) can
emerge and lead to outbreaks of paralytic polio as well as
asymptomatic transmission in communities with a high percent-
age of undervaccinated children. Using data from the World
Health Organization Polio Information System and Global
Polio Laboratory Network, this report describes global polio
outbreaks due to cVDPVs during January 2023–June 2024
and updates previous reports. During the reporting period,
```

pdfplumber, tuned (Cluster A, after tuning):

```
Abstract
Circulating vaccine-derived polioviruses (cVDPVs) can
emerge and lead to outbreaks of paralytic polio as well as
asymptomatic transmission in communities with a high percent-
age of undervaccinated children. Using data from the World
Health Organization Polio Information System and Global
Polio Laboratory Network, this report describes global polio
outbreaks due to cVDPVs during January 2023–June 2024
and updates previous reports. During the reporting period,
```

pdfplumber, default (Cluster C):

```
Abstract                                              vaccine-derived poliovirus
(cVDPVs)* outbreaks occur when
Circulating vaccine-derived polioviruses (cVDPVs) can OPV-related strains
undergo prolonged circulation in com-
emerge and lead to outbreaks of paralytic polio as well as munities with very
low immunity against polioviruses, and the
asymptomatic transmission in communities with a high percent- genetically
reverted virus has regained neurovirulence (vaccine-
age of undervaccinated children. Using data from the World derived poliovirus
[VDPV] emergence) (2,3). After declaration
Health Organization Polio Information System and Global of wild poliovirus
type 2 eradication in 2015, and in an effort
Polio Laboratory Network, this report describes global polio to lower the risk
for cVDPV type 2 (cVDPV2) outbreaks,
outbreaks due to cVDPVs during January 2023–June 2024 immunization programs in
countries using OPV switched
```

Both pdfplumber configurations are calling the same C library on the same PDF; only `x_tolerance=2, y_tolerance=2, use_text_flow=True` separates them. The default heuristic emits every left-column line followed by the same vertical position from the right column — destroying the abstract's reading order, while preserving every word.

B.2 L001 — SCOTUS Slip Opinion 23-477 (single-column prose, page 1)

The same comparison on a single-column SCOTUS opinion. Every parser produces essentially the same text; reader can verify the cluster boundaries collapse on layouts of this shape. Excerpts in `results/specimen_pages/L001_page1_*.txt`. Inter-parser CER across all 5 configurations on this page is below the normalization floor.

B.3 Reproduction

`scripts/specimen_pages.py` re-generates this appendix from the corpus and the parser registry.

Appendix C — Bootstrap percentile-p-value caveat

The two-sided p-value reported is `2 × min( P(bootstrap mean diff ≤ 0), P(bootstrap mean diff ≥ 0) )`, clipped to `[1 / n_resamples, 1]`. This is a conservative percentile-based test, not the

most powerful test for paired data. We use it because (i) it matches the percentile bootstrap CI, (ii) the effect sizes in question are far larger than any test's resolution at n = 30, and (iii) it requires no parametric assumptions beyond exchangeability of the per-doc differences.